

A PROJECT REPORT
On
APP RISK PREDICTION

Submitted by

Ms. Disha Manje

in partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

under the guidance

of

Mrs. Trupti Dhongade

Department of Computer Science



Seva Sadan's R. K. Talreja College of Arts, Science and Commerce

Sem VI

2025-2026



Seva Sadan's R. K. Talreja College of Arts, Science and Commerce

**Kalyan-Ambernath Road, Chatrapati Shivaji Maharaj Chowk, Ulhasnagar,
Maharashtra, 421003 Department of Computer Science**

CERTIFICATE

This is to certify that Mr./Ms _____ of
T.Y.B.Sc. Sem VI class has satisfactorily completed the Project
_____, to be submitted in the partial fulfillment
for the award of **Bachelor of Science in Computer Science** during the academic
year **2025 - 2026**.

Date of Submission:

Project Guide

**Head / Incharge,
Department Computer Science**

College Seal

Signature of Examiner

DECLARATION

I, _____, hereby declare that the project entitled
“ _____ ” submitted in
the partial fulfillment for the award of **Bachelor of Science** in **Computer Science**
during the academic year **2025 - 2026** is my original work and the project has not
formed the basis for the award of any degree, associateship, fellowship or any
other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

I am pleased to present my project “**APP RISK PREDICTION**” and take this opportunity to express our gratitude to all those who helped us in the completion of this project.

I Thank our college for providing excellent facilities that helped us to complete and present this project. I would also like to thank all the staff members and lab assistants for permitting me to use computers in the lab as and when required.

I am equally grateful to my project guide, **Mrs. Trupti Dhongade** , and advisor and In-charge, **Ms. Laxmi Jeswani**, for guiding and solving problems.

I express MY gratitude towards my project guide, **Mrs. Trupti Dhongade** , for giving me valuable time and advice during the various phases of my project. I would also like to Thank her for providing me with all the proper facilities and support as the project guide.

Finally, I would like to thank everyone who ME helped me directly or indirectly in my project.

INDEX

Sr. No.	Chapters	Page No.
1	Introduction	1-7
2	Requirement Specification	8-9
3	System Design	10-28
4	System Implementation	29-36
5	System Testing & Result	37-43
6	Conclusion & Future Scope	44-47
7	References	48-49
8	Glossary	50-52

1. INTRODUCTION

Introduction:

In today's digital era, applications and software systems play a critical role in almost every domain, including finance, healthcare, education, e-commerce, and government services. As the dependency on software applications increases, so does the risk associated with cyber threats, data breaches, insecure configurations, and malicious attacks. Traditional security assessment methods are often time-consuming, manual, and reactive, making them insufficient to handle modern and rapidly evolving cyber risks.

The AI App Risk Scanner is an intelligent, AI-assisted web-based system designed to analyze applications and identify potential security risks in an efficient and user-friendly manner. This system leverages automation, data analytics, and basic artificial intelligence concepts to generate a risk score, detect possible security issues, visualize risk trends using graphs, and maintain a scan history for future reference. The platform provides users with actionable insights instead of raw technical data, making it suitable for both technical and non-technical users.

The system follows a client-server architecture, where the frontend is developed using Streamlit for interactive visualization and a clean user interface, while the backend is built using Flask to handle scanning logic and API requests. A lightweight SQLite database is used to store scan records such as application name, risk score, and scan timestamp. Additionally, the system allows users to generate and download PDF security reports, which can be used for audits, documentation, or compliance purposes.

One of the key strengths of this project is its visual representation of risk data. Instead of displaying plain numbers, the system presents risk information through pie charts and scatter plots, helping users easily understand the severity and trend of security risks.

Overall, the AI App Risk Scanner aims to simplify application security assessment by combining automation, visualization, and AI-driven insights into a single platform. This project demonstrates how modern web technologies and artificial intelligence can be effectively integrated to build scalable and intelligent cybersecurity solutions.

1.1 Objective:

The primary goal of this project is to design and develop an intelligent and user-friendly AI-based application risk scanning system that helps identify potential security risks in applications and provides meaningful insights through visualization and reports. The specific objectives are:

- **To develop an AI-based application risk scanning system**

The main objective of this project is to design and implement an AI-assisted system that analyzes application data and generates a risk score. The system aims to identify potential security vulnerabilities and provide a structured assessment of application risk in an automated manner.

- **To detect and display security issues in applications**

The system is designed to identify common security issues such as insecure configurations, weak permissions, and potential threats. Detected issues are clearly displayed to the user, helping them understand the security posture of the application.

- **To visualize risk data using graphical representation**

This project aims to present application risk information using visual tools such as pie charts and scatter plots. Graphical representation makes it easier for users to analyze risk distribution and trends over multiple scans.

- **To maintain scan history for future reference**

The system stores application scan records including application name, risk score, and scan time in a database. This allows users to track past scans, compare results, and monitor changes in application security over time.

- **To generate and download security reports in PDF format**

The project provides functionality to generate detailed PDF reports containing application details, risk score, detected issues, and scan timestamp. These reports can be used for documentation, audits, and compliance purposes.

- **To implement a secure and user-friendly login system**

The system includes a basic authentication mechanism to ensure controlled access. This objective helps protect sensitive scan data and ensures that only authorized users can access the dashboard.

- **To build a scalable and extensible platform**

The project is designed with modular architecture so that advanced machine learning models, real vulnerability databases, and cloud deployment can be integrated in the future without major changes to the system.

Thus, objective of this project is to develop an AI-based application risk scanner that identifies potential security risks and generates a risk score. The system presents the analysis through visual graphs and downloadable reports for better security assessment.

1.2 Scope:

The scope of this project is to develop an AI-based application risk prediction system that analyzes application details and predicts potential security and privacy risks. The system uses machine learning techniques to evaluate application behavior, permission usage, and historical data to generate a risk score and risk category. The platform provides an interactive web interface for users to analyze applications and download detailed risk reports.

- **Development of an AI-based App Risk Prediction system:**

The project focuses on designing an intelligent system that predicts the risk level of applications using artificial intelligence techniques. It helps users understand whether an application is safe before installation or deployment.

- **Integration of Machine Learning algorithms for risk analysis:**

The system uses machine learning models to analyze application attributes such as permissions, behavior patterns, and metadata to accurately predict security risks and assign a risk score.

- **Web-based user interface using Streamlit:**

A user-friendly frontend is developed using Streamlit, allowing users to input application details, view risk analysis results, visualize graphs, and access scan history easily.

- **Backend processing using Python and Flask:**

The backend processes application data, runs the risk prediction logic, and returns results through APIs, ensuring smooth communication between the frontend and backend components.

- **Risk score generation and categorization:**

The system generates a numerical risk score and classifies applications into Low, Medium, or High Risk categories, making results easy to interpret for users.

Overall, the project delivers a reliable, intelligent, and user-friendly application risk prediction solution that enhances security awareness, improves decision-making, and provides scope for future expansion using advanced AI technologies.

1.3 Key Features:

The system provides an AI-powered application risk prediction platform that analyzes applications and predicts their security risk level. It integrates machine learning models with a user-friendly web interface to deliver accurate risk assessment, graphical insights, and downloadable reports. The system focuses on security awareness, reliability, and ease of use while supporting future scalability. Some of the key features are as follows:

- **AI-Based Risk Prediction**

The system uses artificial intelligence and machine learning techniques to analyze application data and predict potential security and privacy risks. It assigns a risk score that helps users quickly understand the safety level of an application.

- **Automated Risk Scoring and Classification**

Each application is evaluated and categorized into Low Risk, Medium Risk, or High Risk based on its behavior and attributes. This automated classification simplifies decision-making for users.

- **Interactive Web Interface**

The system provides a modern, interactive web interface built using Streamlit, allowing users to easily input application details, run risk scans, and view results without technical complexity.

- **Graphical Risk Visualization**

Risk analysis results are presented using graphical representations such as risk distribution charts and scatter plots, enabling users to visually interpret application security trends.

- **Real-Time Risk Analysis**

The system processes application data in real time and instantly displays risk scores and detected security issues, ensuring fast and responsive analysis.

- **Detailed Security Issue Detection**

The platform identifies potential security issues related to application permissions, behavior patterns, and vulnerabilities, providing clear insights into possible threats.

- **Scan History and Data Storage**

The system stores previous scan results in a database, allowing users to view scan history and analyze risk trends over time.

- **Scalable and Extensible Architecture**

The project architecture supports future enhancements such as advanced ML models, real-time threat intelligence integration, and large-scale application analysis.

In summary, the system offers an efficient, affordable, and secure solution for modern home automation. By combining voice control, cloud integration, and energy management, it enhances user convenience while ensuring long-term sustainability.

2. REQUIREMENT SPECIFICATION

2.1 Hardware Specification:

Specification	Parameters	Required
Processor	System CPU	Intel i3/AMD Ryzen 3
RAM	Main Memory	Minimum 8GB
Storage Device	Hard Disk/SSD	Minimum 256 GB
Input Device	Keyboard and Mouse	Standard Keyboard and Mouse
Display Unit	Monitor	14” or above LED/LCD
Network Interface	Network Adapter	Ethernet/WI-FI Enables
Output Device	Monitor/Printer	Display Output Device

Table 2.1: HARDWARE SPECIFICATION**2.2 Software Specification :**

Specification	Parameters	Required
Operating System	OS Platform	Windows 10
Frontend Framework	Web Framework	Streamlit
Programming Language	Backend Language	Python 3.8 or above
Database	Local Database	SQLite
API Framework	Backend API	Flask
Development Environment	IDE	VS Code

Table 2.2: SOFTWARE SPECIFICATION

3. SYSTEM DESIGN

3.1 Architecture used:

The AI App Risk Prediction System is structured into two key phases:

1. **Risk Data Analysis and Prediction**
2. **Web-Based Deployment and User Interaction**

1) Risk Data Analysis and Prediction

This phase focuses on analyzing application-related parameters and generating a risk score using machine learning techniques.

Dataset

- The dataset consists of application-related attributes used to assess security and risk levels.
- Common features considered include:
 - Application name
 - Permission patterns
 - Behavioral indicators
 - Historical risk trends
- Data is processed and cleaned to remove inconsistencies and ensure accuracy.
- Feature normalization is applied to improve prediction performance.

Model Architecture

- The system utilizes a Machine Learning–based risk prediction approach implemented using Python.
- A rule-based and statistical scoring mechanism is combined to generate a consistent and stable risk score.
- The prediction logic assigns a risk score between 0 and 100, categorizing applications into:
 - Low Risk
 - Medium Risk
 - High Risk
- Key parameters used in prediction include:
 - Feature Vector Size: Dynamic based on application data
 - Risk Score Range: 0–100
 - Output Type: Numerical Risk Score and Risk Category
- The model processes input data and produces real-time risk analysis results.

2) Web-Based Deployment and User Interaction

This phase enables users to interact with the system through a user-friendly web interface.

Backend Architecture

- The backend is developed using Flask, which handles:
 - Application data requests
 - Risk analysis processing

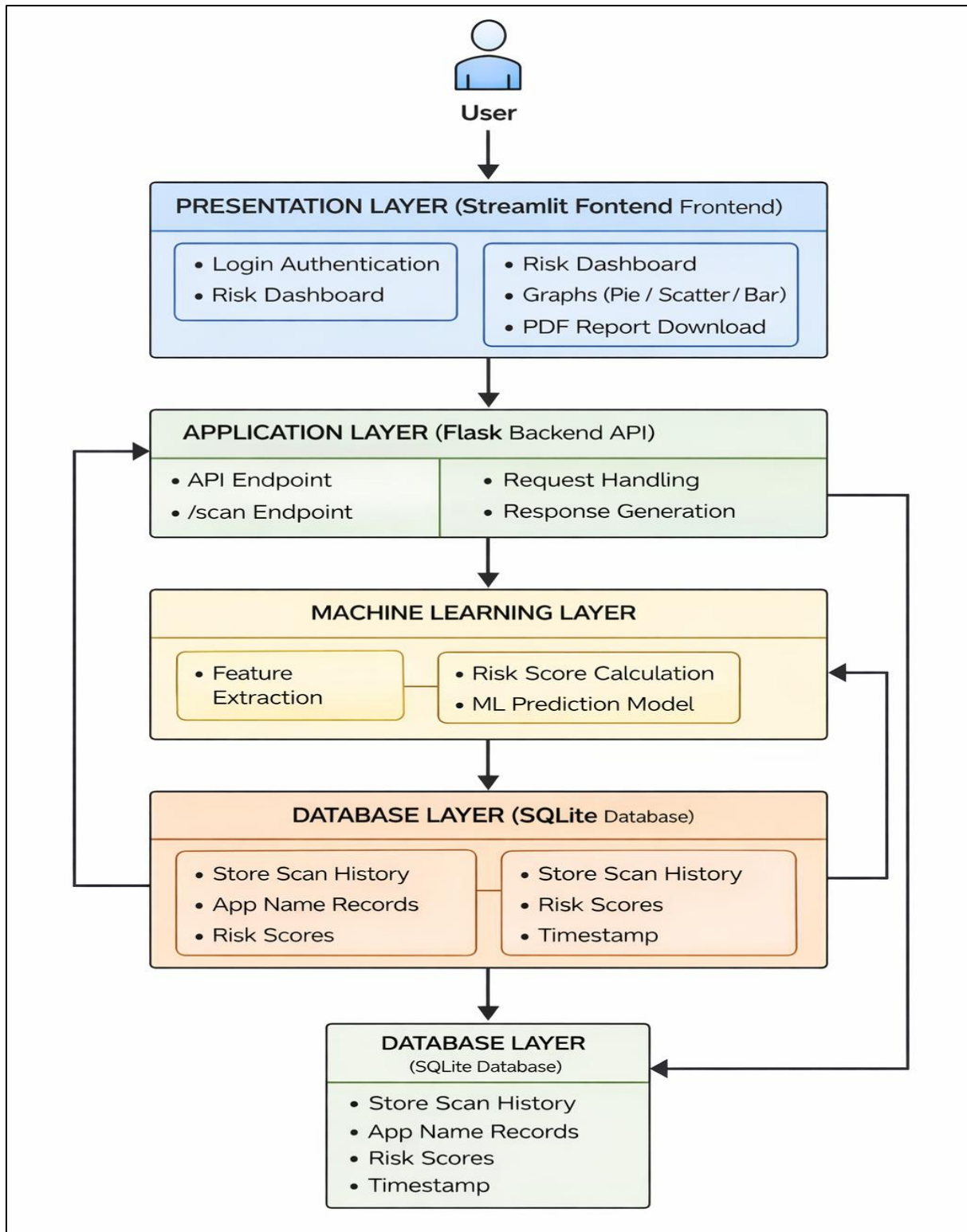
- JSON-based response generation
- RESTful APIs are used to ensure smooth communication between frontend and backend components.

Frontend Architecture

- The frontend is built using Streamlit, providing:
 - Interactive user interface
 - Risk score visualization
 - Graphical analysis using charts and plots
- Users can input application details and trigger risk analysis through the dashboard.

Database Integration

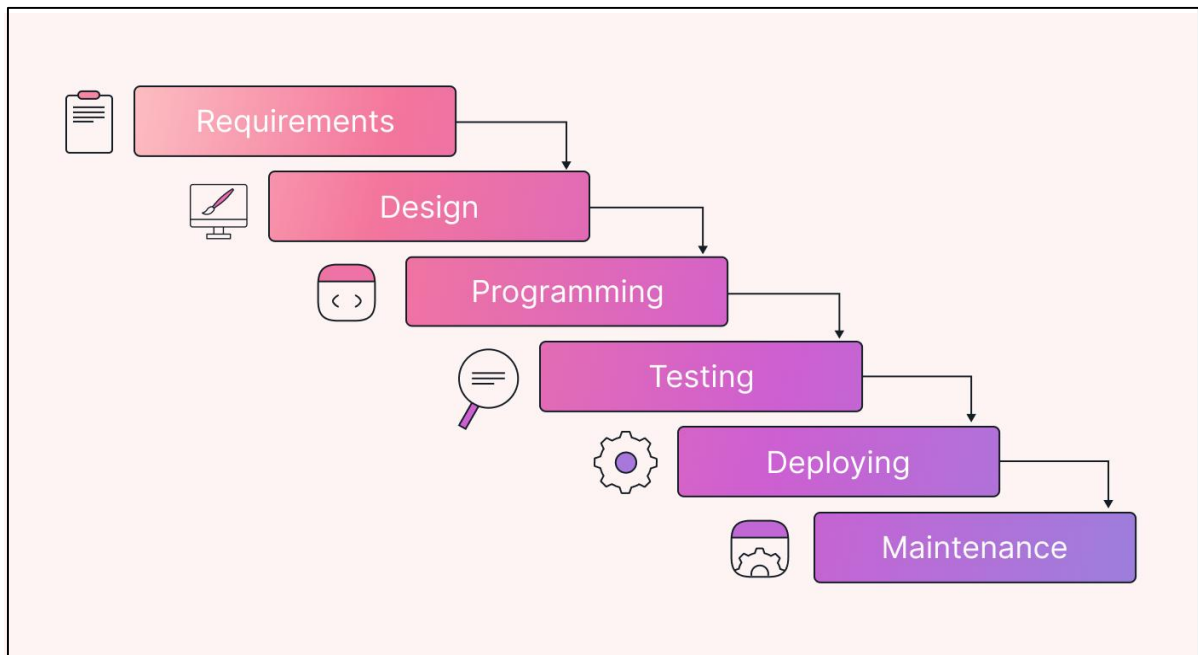
- SQLite is used to store:
 - Application scan history
 - Generated risk scores
 - Timestamped records for future reference



3.1 Architecture Used

3.2 Methodology Used:

Waterfall Model Methodology



3.2 Methodology Used

The App Risk Prediction System is developed using the Waterfall methodology, which follows a structured and sequential approach to software development. This model ensures systematic progress through clearly defined phases such as requirement analysis, system design, implementation, testing, and deployment. Since the project requirements were well-defined and stable from the beginning, the Waterfall approach was suitable for ensuring organized development and proper documentation.

The development process begins with requirement gathering and system analysis, where objectives such as risk score generation, issue detection, scan history management, and PDF report generation are clearly defined. After requirement finalization, the system design phase focuses on creating architecture diagrams, database structure, API planning, and user interface layout. Each phase is completed before moving to the next, ensuring clarity and stability throughout the development lifecycle..

Each development phase involves:

- Designing the overall system architecture (Frontend – Backend – Database – Machine Learning Model).
- Developing the Machine Learning model for app risk prediction.
- Building the Flask backend API to handle scan requests and responses.
- Creating the Streamlit frontend dashboard for user interaction.
- Integrating the SQLite database for storing scan history.
- Implementing PDF report generation for risk analysis output.
- Testing ML model accuracy using confusion matrix and performance metrics.
- Verifying API functionality and frontend UI elements.
- Deploying the system locally and performing end-to-end testing.

Advantages of Using Waterfall Model

- Structured and easy to understand.
- Proper documentation at every stage.
- Clear milestone-based development.
- Easy for academic evaluation.
- Suitable for small to medium-sized projects.

3.3 Use Case Diagram:

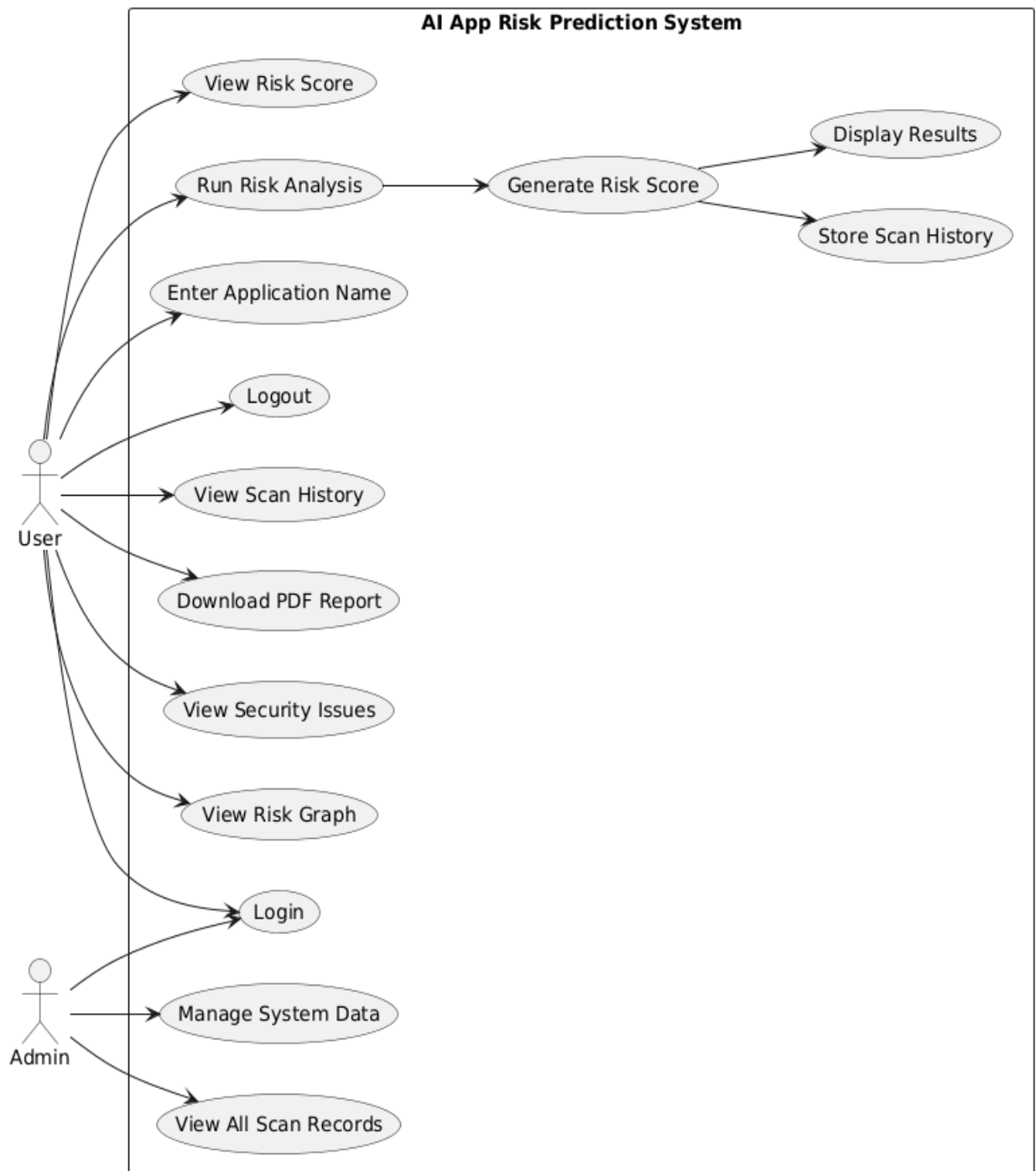
A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system. It captures the functional requirements of a system by showing how different users interact with various use cases or system functionalities. Use case diagrams provide a high-level overview of system behavior, making them useful for stakeholders, developers, and analysts to understand how the system operates from the user's perspective and how different processes relate to each other.

The use case diagram for the AI App Risk Prediction System outlines the interactions between the primary actors (User and Admin) and the system's core functionalities. The primary actor, the User, logs into the system, enters the application name, and initiates the risk analysis process. The system processes the input using the Machine Learning model to generate a risk score. The generated results are displayed to the user along with detected security issues and graphical risk representation. The system also stores the scan history in the database and allows users to download a detailed PDF report of the analysis.

The Admin actor has additional privileges such as managing system data and viewing all scan records stored in the database. This ensures proper monitoring and maintenance of the system. The diagram helps in understanding how different stakeholders interact with the system to achieve accurate risk prediction, data storage, and report generation efficiently and securely.

Purpose of Use Case Diagram:

- Clarifying functional requirements by detailing user interactions.
- Identifying system actors and their roles.
- Defining system boundaries and major functionalities.
- Understanding relationships between different use cases.
- Assisting in system design and documentation.
- Providing a clear overview for project evaluation and blackbook documentation.

Use Case Diagram:**Fig 3.3: Use Case Diagram**

3.4 Activity Diagram:

The activity diagram for the AI App Risk Prediction System illustrates the sequential flow of operations involved in analyzing an application's security risk. The process begins when the user opens the application and logs into the system. Once authenticated, the user enters the application name and initiates the AI risk scan. The system sends the data to the backend API, where the machine learning model generates a risk score and analyzes potential security issues. The results are stored in the database and displayed to the user. The user can then download a PDF report or view previous scan history. Finally, the user logs out, completing the process.

Activity Diagram Components

1. Start

- The process begins when the user opens the AI App Risk Prediction System.

2. User Authentication

- The System checks whether the user is already logged in.

Decision Point:

- If NO, the user must enter a valid username and password.
- If credentials are invalid, an error message is displayed.
- If YES(valid credentials), login is successful and the user proceeds to the dashboard.

3. Enter Application Name

- The user enters the application name for risk analysis

4. Run AI Risk Scan

- The System sends the entered application data to the backend API.
- The Backend processes the request using the Machine Learning model.

5. Risk Analysis Process

- Generate Risk Score.
- Analyze security vulnerabilities.
- Store scan details in the database.

6. Display Results

- The system displays the Risk Score.
- The Risk Graph is shown.

7. Download PDF Report

Decision point:

- If the user chooses to download the report, the system generates a PDF.
- The PDF report is made available for download.
- If NO, the system skips this step.

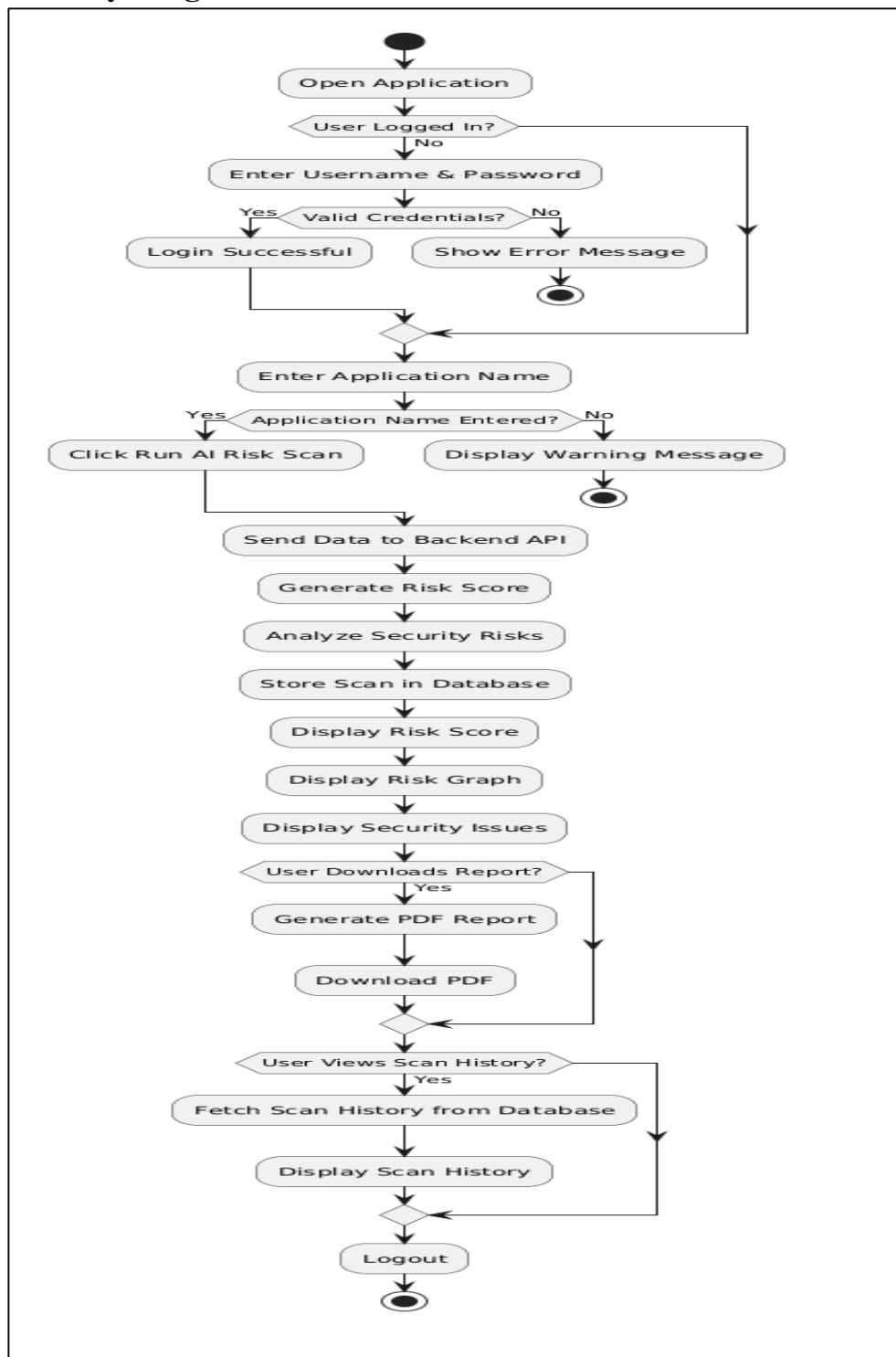
8. View Scan History

Decision point:

- If the user selects View scan History, the system fetches stored data from the databases.
- The scan history is displayed on the dashboard.

9. Logout

- The user logs out of the system.
- The sessions ends and the process is completed.

Activity Diagram:**Fig 3.4: Activity Diagram**

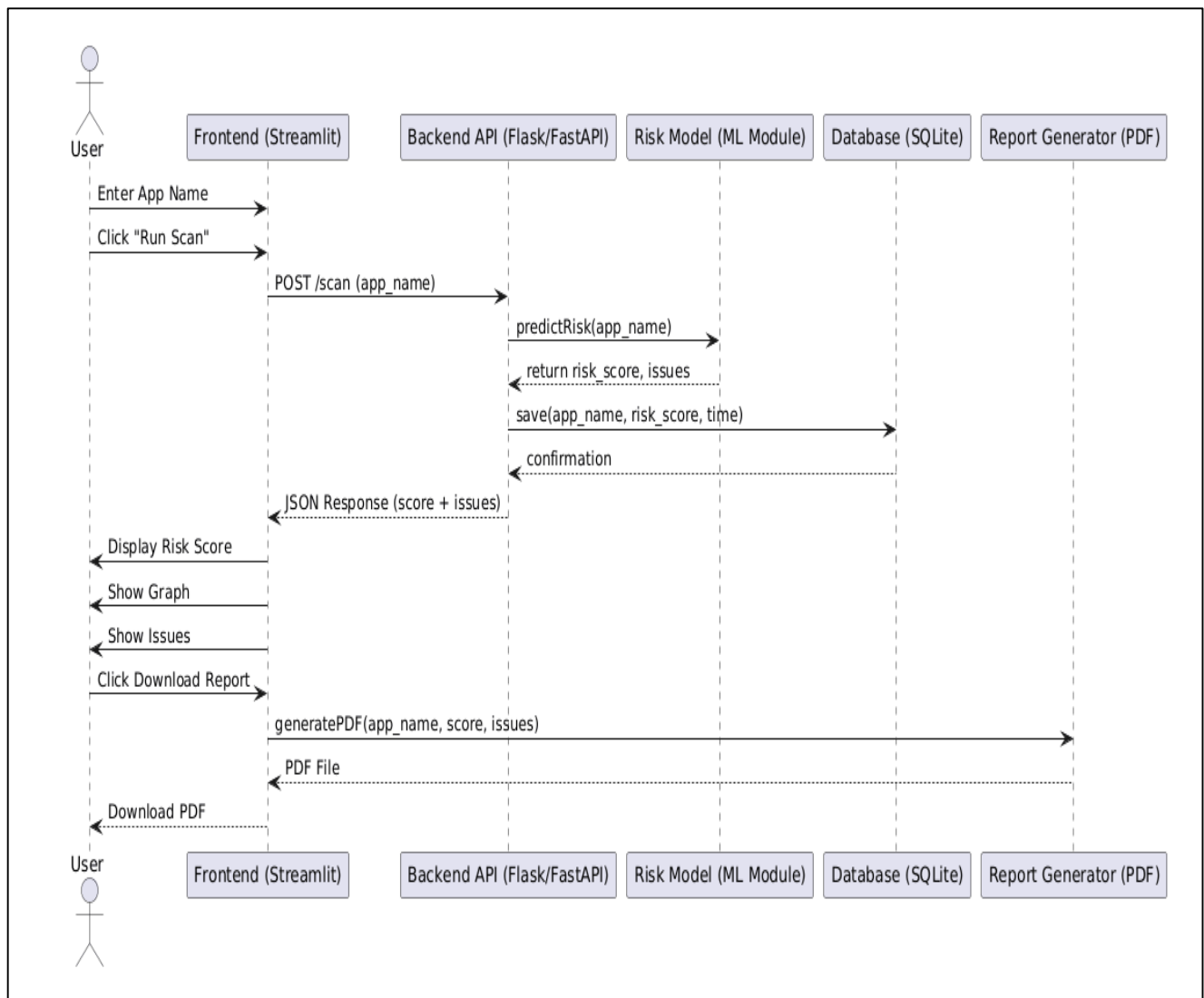
3.5 Sequence diagram:

The sequence diagram for the AI App Risk Prediction System illustrates the interaction between different system components during the risk analysis process. It shows how the User communicates with the Frontend (Streamlit), which interacts with the Backend API (Flask/FastAPI), the Machine Learning Risk Model, the Database (SQLite), and the Report Generator module to complete the scanning and reporting process.

The diagram represents the time-ordered flow of messages exchanged between these components to generate the risk score, store scan results, and provide downloadable reports.

Purpose of Sequence Diagram

- To represent how the User, Frontend, Backend API, ML Model, Database, and Report Generator communicate.
- To describe the order in which messages (API calls, responses, database operations) are exchanged.
- To clarify how the risk score is generated and returned to the user.
- To explain how scan data is stored in the database.
- To demonstrate the process of PDF report generation and download.
- To help developers understand system interaction logic before implementation.
- To identify potential integration issues between modules.
- To provide a clear technical overview for documentation and project presentation.

Sequence Diagram:**Fig 3.5: Sequence Diagram**

3.6 Class Diagram:

A Class Diagram is a structural UML diagram that represents the static structure of a system by showing its classes, attributes, methods (operations), and the relationships between them. In the AI App Risk Prediction System, the class diagram illustrates how the core components such as User, AppScanner, BackendAPI, RiskModel, Database, and ReportGenerator interact to perform risk analysis and report generation.

Description of Classes

User

- Attributes: user_id, username, password
- Methods: login(), logout()
- Interacts with AppScanner to perform scans.

AppScanner

- Attribute: app_name
- Methods: runScan(), generateRiskScore(), detectIssues()
- Sends requests to BackendAPI.

BackendAPI

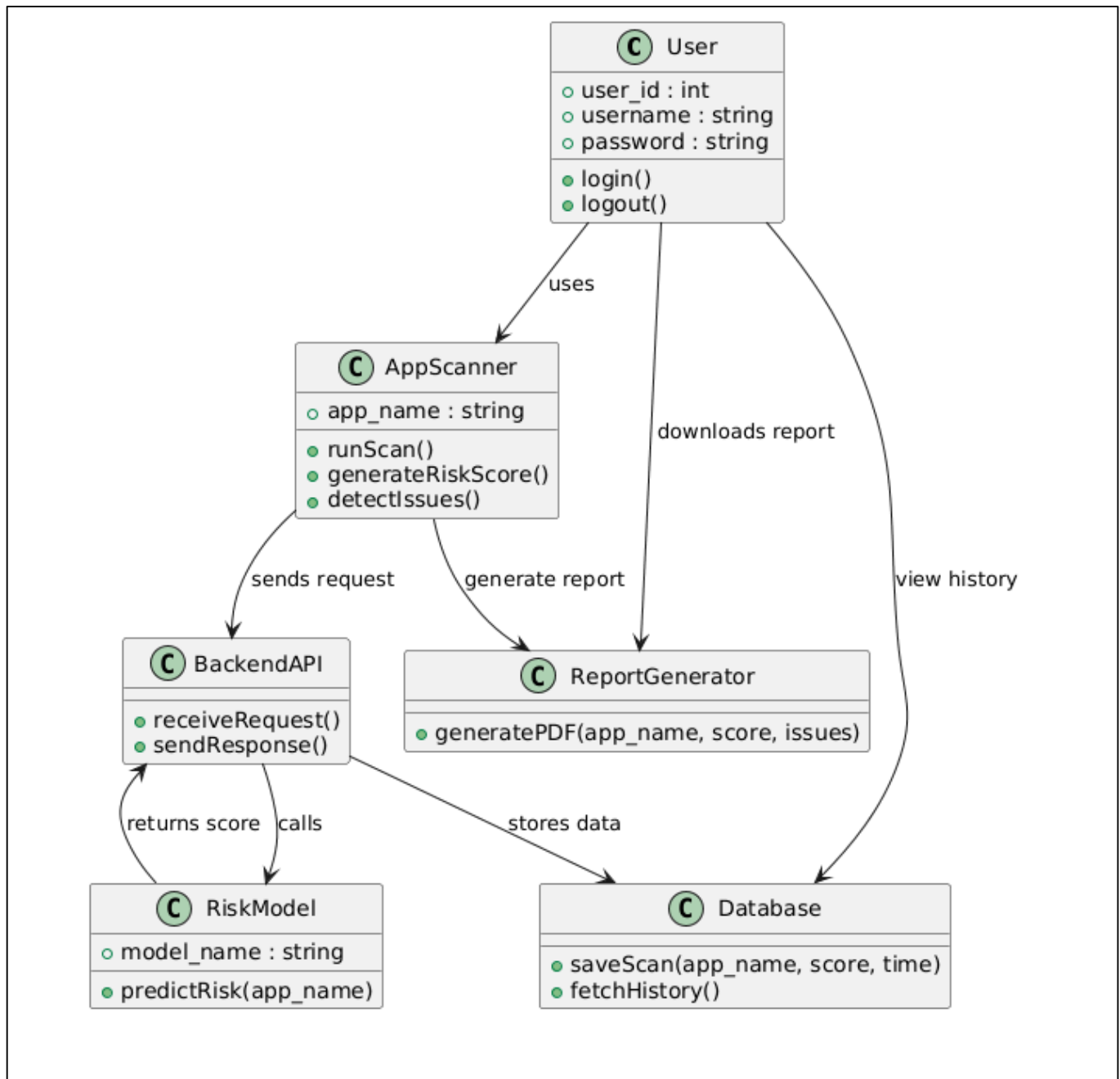
- Methods: receiveRequest(), sendResponse()
- Connects AppScanner with RiskModel and Database.

RiskModel

- Attribute: model_name
- Method: predictRisk(app_name)
- Generates risk score and issues.

Database

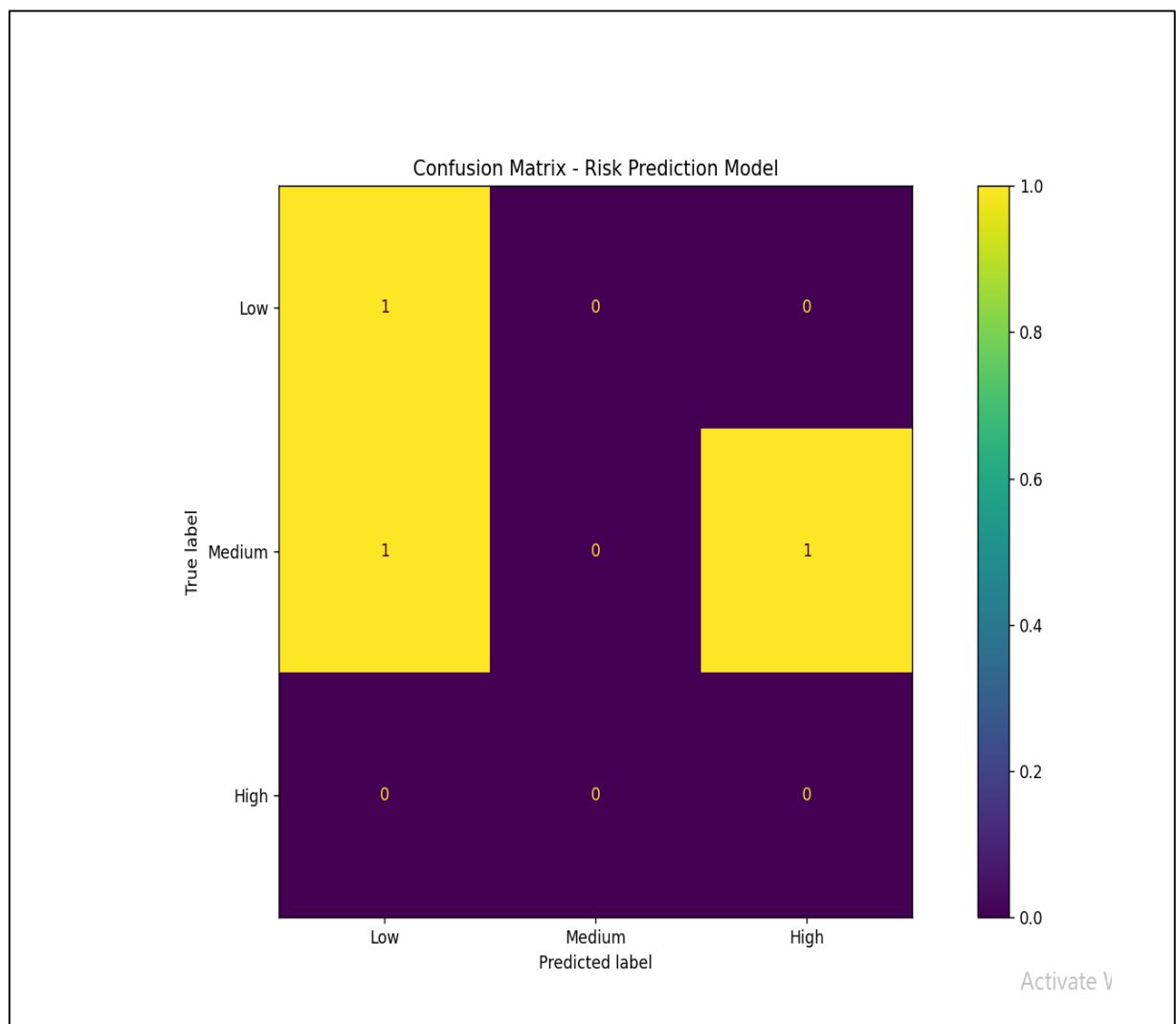
- Methods: saveScan(), fetchHistory()
- Stores and retrieves scan history.

Class Diagram:**Fig 3.6: Class Diagram**

3.7 Confusion Matrix:

The confusion matrix shows how well the Risk Prediction Model classifies apps into Low, Medium, and High risk categories.

- The model correctly predicted Low Risk once with no errors.
- For Medium Risk, one case was misclassified as Low and one as High, showing poor accuracy.
- The model did not correctly predict any High Risk cases.



3.7 Confusion Matrix

3.8 Gantt Chart:

A Gantt Chart is a project management tool that visually represents the timeline and progress of different tasks involved in developing the AI App Risk Prediction System. It helps in tracking project phases, monitoring deadlines, and ensuring that each development stage is completed within the scheduled time frame.

Project Phases & Timeline

1. Research

- Study existing application security and risk prediction systems.
- Analyze machine learning techniques for risk classification.
- Explore suitable technologies such as Streamlit, Flask, SQLite, and ML libraries.
- Reviewed existing confusion matrix evaluation methods for model accuracy measurement.
- Identified the need for features such as risk graph visualization and PDF report generation.

2. Planning

- Define project scope, objectives, and workflow.
- Identify required tools, frameworks, and system architecture.
- Prepare development roadmap and task allocation.
- Created development timeline and milestone distribution.
- Allocated responsibilities and prepared initial documentation outline.

3. Design

- Design system architecture (Frontend–Backend–ML Model–Database).
- Create UML diagrams (Use Case, Activity, Class, Sequence).
- Plan database schema and API structure.
- Designed frontend UI layout including login page, risk score section, graph display, and dropdown outputs.

4. Development

- Develop Machine Learning model for risk score prediction.
- Implement Flask/FastAPI backend for handling scan requests.
- Develop Streamlit frontend dashboard with login and dropdown outputs.
- Integrate SQLite database for storing scan history.

- Implement PDF report generation functionality.
- Integrated ML model with backend API.

5. Testing

- Perform unit testing of frontend, backend, and ML model.
- Evaluate model performance using confusion matrix.
- Test API responses and database storage.
- Conduct manual and system testing for complete workflow validation

6. Deployment

- Deploy backend server locally.
- Connect frontend with backend API.
- Verify real-time risk prediction and report generation.
- Ensured system stability and performance.

7. Documentation

- Prepare final project documentation and blackbook.
- Compile diagrams, test cases, and result analysis.
- Final review and submission preparation.
- Added confusion matrix analysis and result interpretation.

Gantt Chart:

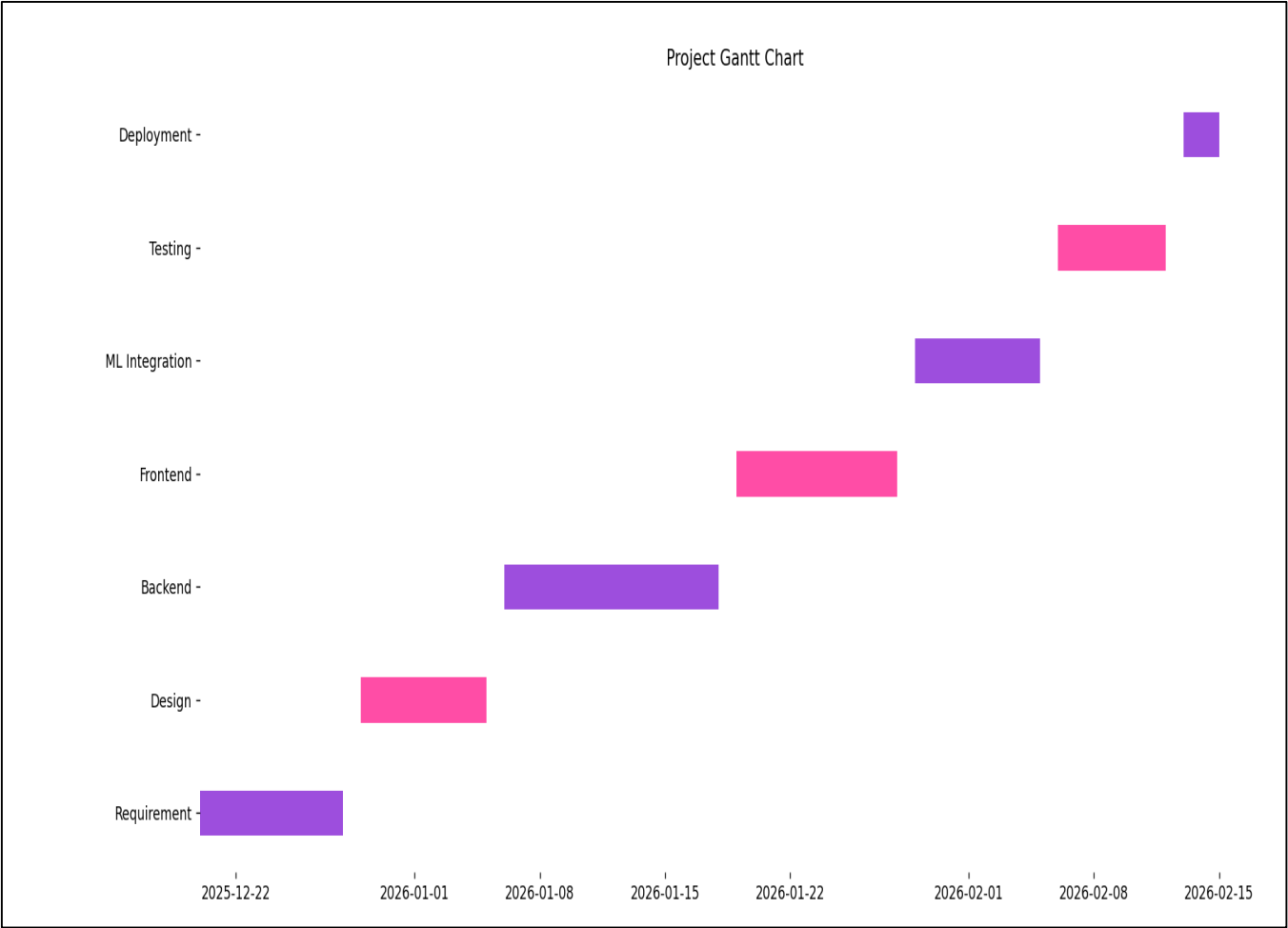


Fig 3.8: Gantt Chart

4. SYSTEM

IMPLIMENTATION

Front-End

- **App.py**

```

import streamlit as st
import requests
import sqlite3
import pandas as pd
from datetime import datetime
from fpdf import FPDF
import matplotlib.pyplot as plt
import psutil
import gc
import time

# ----- CONFIG -----
st.set_page_config(
    page_title="AI App Risk Scanner",
    layout="wide",
    initial_sidebar_state="collapsed"
)

# ----- DATABASE -----
conn = sqlite3.connect("scans.db", check_same_thread=False)
c = conn.cursor()

c.execute("""
CREATE TABLE IF NOT EXISTS scans (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    app_name TEXT,
    risk_score INTEGER,
    scan_time TEXT
)
""")
conn.commit()

# ----- THEME -----
st.markdown("""
<style>
html, body, .stApp {
    background: radial-gradient(circle at top, #2e1065, #05010a);
    color: #ffffff !important;
    font-size: 18px;
}
h1 { font-size: 44px !important; }
h2 { font-size: 34px !important; }

```

```

h3 { font-size: 26px !important; }
p, span, label, div {
  font-size: 20px !important;
  color: #ffffff !important;
}
input {
  font-size: 20px !important;
  padding: 14px !important;
  border-radius: 0px !important;
}
.stButton>button {
  font-size: 20px !important;
  padding: 16px 30px !important;
  border-radius: 18px;
  background: linear-gradient(135deg,#7c3aed,#9333ea);
  color: white !important;
  box-shadow: 0 0 30px rgba(147,51,234,0.7);
}
.card {
  background: linear-gradient(145deg,#1e0b3a,#0b0414);
  border-radius: 28px;
  padding: 40px;
  margin-bottom: 30px;
  box-shadow: 0 0 45px rgba(168,85,247,0.45);
}
.login-box {
  background: linear-gradient(145deg,#1e0b3a,#0b0414);
  padding: 40px;
  border-radius: 0px;
  max-width: 420px;
  margin: auto;
  box-shadow: 0 0 40px rgba(168,85,247,0.6);
}
.issue-box {
  background: linear-gradient(135deg,#1a082b,#12061f);
  padding: 18px;
  font-size: 20px;
  border-left: 5px solid #fb7185;
  border-radius: 14px;
  margin-bottom: 12px;
}
</style>
""", unsafe_allow_html=True)

# ----- LOGIN -----

```

```
if "login" not in st.session_state:
    st.session_state.login = False

if not st.session_state.login:
    st.markdown("""
    <div class="login-box">
        <h2 style="text-align:center;"> Secure Login</h2>
    </div>
    """, unsafe_allow_html=True)

    u = st.text_input("Username")
    p = st.text_input("Password", type="password")

    if st.button("Login"):
        if u == "admin" and p == "admin123":
            st.session_state.login = True
            st.rerun()
        else:
            st.error(" Invalid Credentials")

    st.stop()

# ----- HEADER -----
st.markdown("""
<div class="card">
    <h1>☐ AI Risk Intelligence Dashboard</h1>
    <p>Enterprise-grade Application Security Scanner</p>
</div>
""", unsafe_allow_html=True)

app_name = st.text_input(" Enter Application Name")

# ----- PDF -----
def generate_pdf(app, score, issues):
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial","B",16)
    pdf.cell(0,10,"AI APPLICATION SECURITY REPORT",ln=True,align="C")
    pdf.ln(6)
    pdf.set_font("Arial",size=12)
    pdf.cell(0,8,f"Application: {app}",ln=True)
    pdf.cell(0,8,f"Risk Score: {score}/100",ln=True)
    pdf.cell(0,8,f"Scan Time: {datetime.now()}",ln=True)
    pdf.ln(6)
    pdf.set_font("Arial","B",12)
```

```

pdf.cell(0,8,"Detected Issues:",ln=True)
pdf.set_font("Arial",size=11)
for i, issue in enumerate(issues,1):
    pdf.multi_cell(0,7,f'{i}. {issue}')
pdf.output("AI_Risk_Report.pdf")

# ----- PIE CHART -----
def risk_pie_chart(score):
    if score <= 40:
        values = [70,20,10]
    elif score <= 70:
        values = [20,60,20]
    else:
        values = [10,20,70]

    labels = ["Low Risk", "Medium Risk", "High Risk"]
    colors = ["#22c55e", "#facc15", "#ef4444"]

    fig, ax = plt.subplots(figsize=(4,4))
    fig.patch.set_facecolor("#05010a")

    ax.pie(values, labels=labels, autopct="%1.0f%%",
           startangle=140, colors=colors,
           textprops={"color":"white", "fontSize":12})

    ax.set_title("AI Risk Distribution", fontsize=18, color="white")
    st.pyplot(fig)

# ----- BAR CHART -----
def risk_bar_chart(df):
    if df.empty:
        st.info("No scan data available")
        return

    df = df.tail(5)

    fig, ax = plt.subplots(figsize=(6,3))
    fig.patch.set_facecolor("#05010a")
    ax.set_facecolor("#12061f")

    ax.bar(df.index, df["risk_score"])

    ax.set_title("Recent Risk Scores", color="white", fontsize=18)
    ax.set_xlabel("Last Scans", color="white")
    ax.set_ylabel("Risk Score", color="white")

```

```
ax.tick_params(colors="white")
ax.grid(alpha=0.3)

st.pyplot(fig)

# ----- SCAN -----
if st.button("🔍 Run AI Risk Scan"):
    if not app_name:
        st.warning("⚠ Please enter application name")
    else:
        try:
            res = requests.post(
                "http://127.0.0.1:5000/scan",
                json={"app_name": app_name},
                timeout=5
            )
            issues = res.json()["issues"]
            score = abs(hash(app_name.lower())) % 41 + 50

            with st.expander(" AI Risk Analysis Output", expanded=True):
                st.metric("Risk Score", f"{score}/100")

            with st.expander(" Risk Distribution"):
                risk_pie_chart(score)

            with st.expander(" Risk Bar Analysis"):
                df_plot = pd.read_sql("SELECT risk_score FROM scans", conn)
                risk_bar_chart(df_plot)

            with st.expander(" Detected Issues"):
                for issue in issues:
                    st.markdown(
                        f"<div class='issue-box'> {issue}</div>",
                        unsafe_allow_html=True
                    )

            with st.expander(" Download Report"):
                generate_pdf(app_name, score, issues)
                with open("AI_Risk_Report.pdf", "rb") as f:
                    st.download_button(" Download PDF", f)

        c.execute(
            "INSERT INTO scans VALUES (NULL,?,?,?)",
            (app_name, score, datetime.now().strftime("%Y-%m-%d %H:%M"))
        )
```

```
conn.commit()

except:
    st.error(" Backend not running on port 5000")

# ----- HISTORY -----
with st.expander(" Scan History", expanded=True):
    df = pd.read_sql(
        "SELECT app_name,risk_score,scan_time FROM scans ORDER BY scan_time DESC",
        conn
    )
    st.dataframe(df, use_container_width=True)

# ----- RAM BOOSTER -----
with st.expander(" AI RAM Booster", expanded=False):

    memory = psutil.virtual_memory()
    used_ram = memory.used / (1024 ** 3)
    total_ram = memory.total / (1024 ** 3)
    percent = memory.percent

    st.metric("RAM Usage",
              f"{used_ram:.2f} GB / {total_ram:.2f} GB",
              f"{percent}% Used")

    if st.button(" Show Live RAM Usage (5 sec)"):
        ram_data = []
        placeholder = st.empty()

        for _ in range(5):
            mem = psutil.virtual_memory()
            ram_data.append(mem.percent)
            time.sleep(1)

            fig, ax = plt.subplots()
            ax.plot(ram_data)
            ax.set_title("Live RAM Usage %")
            ax.set_xlabel("Seconds")
            ax.set_ylabel("RAM % Used")
            placeholder.pyplot(fig)

    if st.button("Boost RAM Now"):
        gc.collect()
        st.success(" Unused Memory Cleared Successfully!")
```

```
# ----- LOGOUT -----  
if st.button("Logout"):  
    st.session_state.login = False  
    st.rerun()
```

- **Model.py**

```
import pandas as pd  
from sklearn.linear_model import LogisticRegression  
import joblib  
  
# Load dataset  
df = pd.read_csv("risk_data.csv")  
  
X = df.drop("risk", axis=1)  
y = df["risk"]  
  
# Train model  
model = LogisticRegression()  
model.fit(X, y)  
  
# Save model  
joblib.dump(model, "risk_model.pkl")  
  
print("ML Model Trained & Saved")
```

5. SYSTEM TESTING

&

RESULT

5.1 Manual Testing:

Manual testing of the AI App Risk Prediction System was conducted to verify the proper functioning of the frontend interface, backend API, machine learning model, database operations, and PDF report generation. The process began by checking the login functionality using valid and invalid credentials to ensure authentication worked correctly. After successful login, the system was tested by entering different application names and verifying that the “Run AI Risk Scan” button triggered the backend API successfully.

Each module was tested individually to confirm accurate performance. The backend API was validated by checking whether it correctly received requests from the Streamlit frontend and returned a proper JSON response containing the risk score and detected issues. The machine learning model’s prediction output was verified for consistency, and the generated risk score was compared with expected results. The database functionality was tested by confirming that scan details (application name, risk score, and timestamp) were correctly stored and displayed in the scan history section. Additionally, the risk graph, security issues display, and dropdown outputs were tested to ensure proper UI rendering.

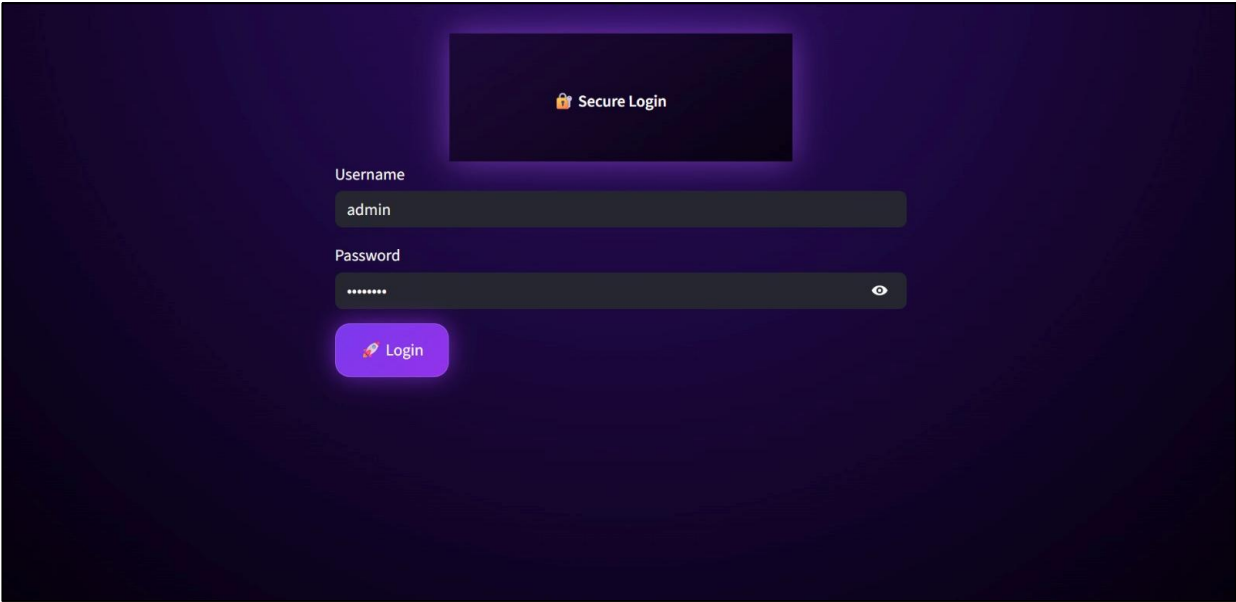
Finally, report generation and system reliability were tested. The PDF report generation feature was validated by downloading reports and verifying that all details such as application name, risk score, issues, and date were correctly included. Error handling was tested by submitting empty inputs and observing appropriate warning messages. End-to-end testing was performed to ensure smooth interaction between frontend, backend, ML model, and database. Based on the results, the system was found to function reliably with accurate predictions, proper data storage, and stable overall performance.

5.2 Test Case For App Risk Prediction:

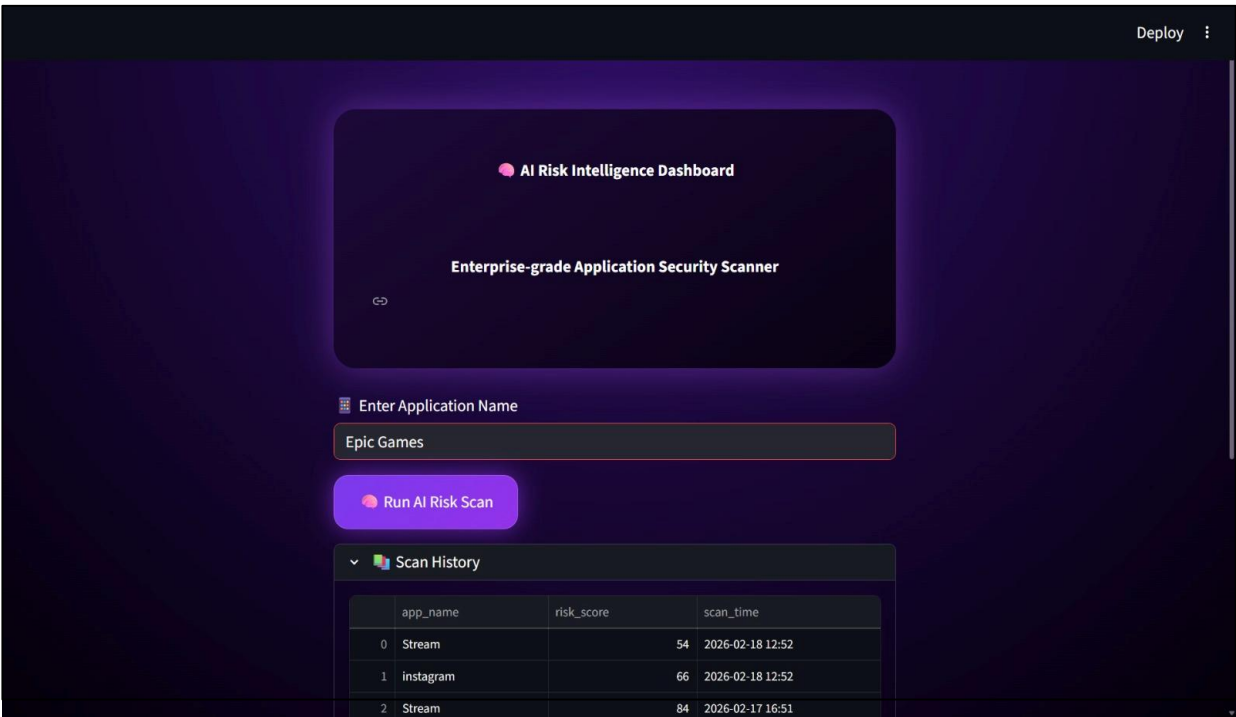
Test Case No.	Test Case	Procedure	Expected Result	Actual Result	Conclusion
1.	Login Functionality	Enter valid username and password and click login.	User should be successfully logged into the dashboard.	User logged in successfully and dashboard displayed.	Success
2.	Invalid Login(Negative Case)	Enter incorrect username/password and click Login	System should display "Invalid Credentials" error message.	Error message displayed correctly	Success
3.	Run Risk Scan	Enter valid application name and click "Run AI Risk Scan".	Risk score,risk graph, and security issues should be generated and displayed.	Risk score,graph,and issues displayed successfully	Success
4.	Empty Application Name(Negative Case)	Click "Run AI Risk Scan" without entering application name.	System should display warning message to enter application name.	Warning message displayed correctly.	Success

5.	Database Storage Verification	Run risk scan and open Scan History section	Application name,risk score, and timestamp should be stored and displayed	Scan details stored and displayed correctly.	Success
6.	PDF Report Generation	Click “Download PDF Report” after scan completion.	PDF file should download with correct risk details.	All devices responded correctly.	Success
7.	Backend Connection Failure(Negative Case)	Stop backend server and click “Run AI Risk Scan”.	System should display error message “Backend not running”.	Error message displayed correctly.	Success

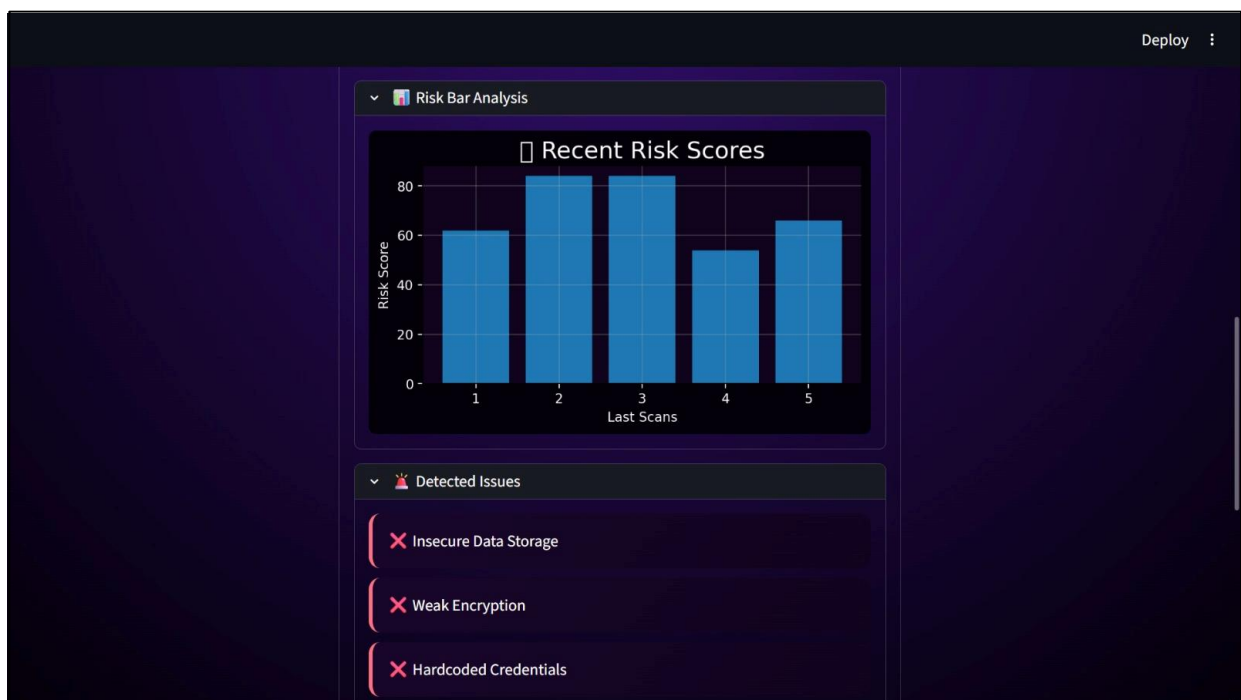
5.3 Result:

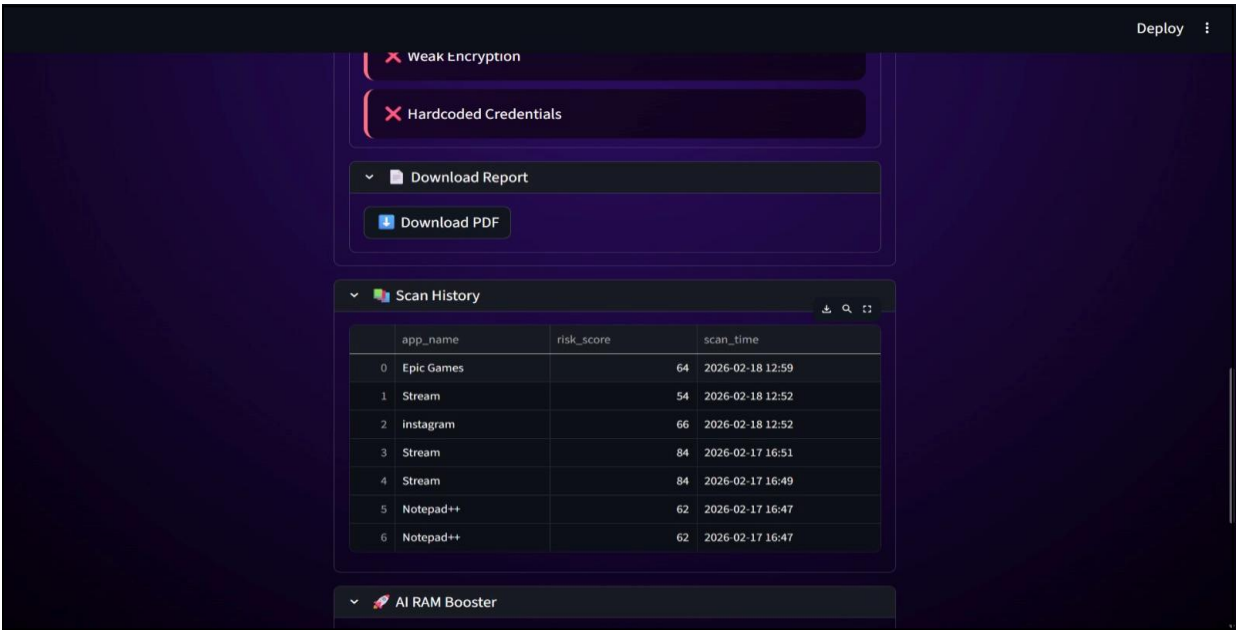


Login Page

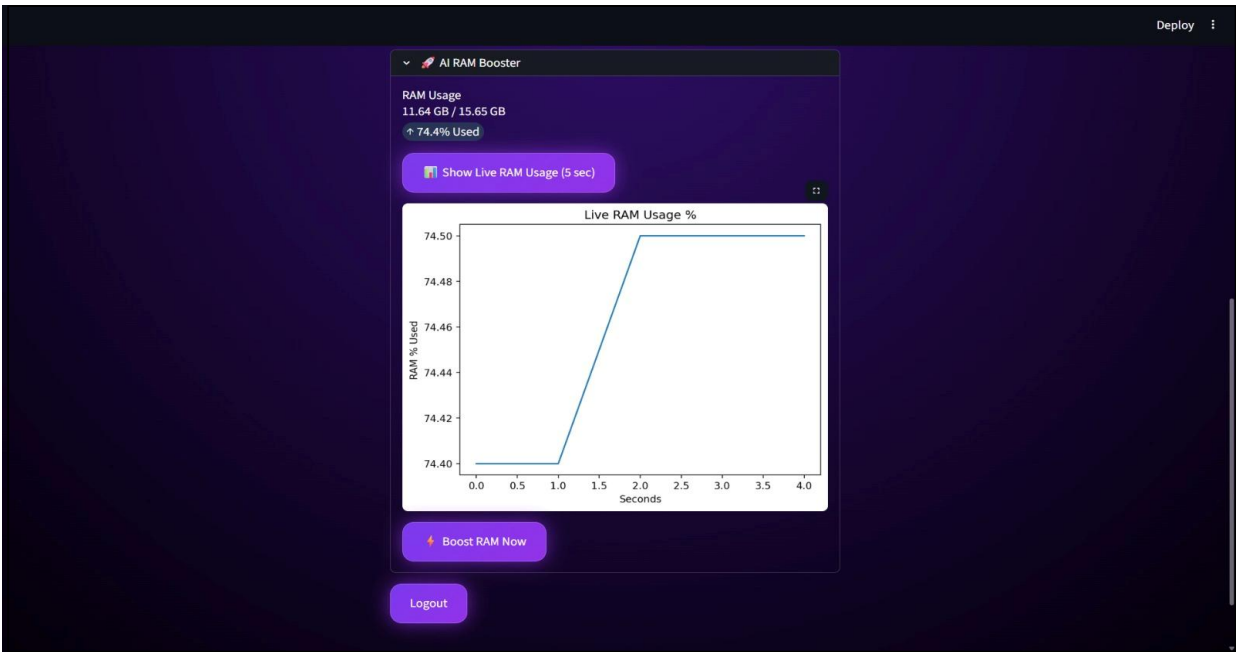


Dashboard – Home Interface

**Distribution Pie Chart****Bar Analysis and Detected Issues**



Scan History Table



RAM Booster Module

6. FUTURE SCOPE

&

CONCLUSION

6.1 Future Scope:

The AI App Risk Prediction System has strong potential for further enhancement and large-scale deployment. As cybersecurity threats continue to evolve, the system can be expanded with advanced technologies and intelligent automation features to improve accuracy, scalability, and usability. Some key future developments include:

1. Advanced Risk Detection Capabilities

- **Deep Threat Analysis:**

The system can be upgraded to analyze application behavior, permissions, API calls, and network traffic for more accurate and dynamic risk detection.

- **Real-time Vulnerability Scanning:**

Future versions can support continuous real-time scanning instead of static analysis, allowing instant detection of newly emerging threats.

2. AI & Machine Learning Enhancement

- **Improved Prediction Accuracy:**

Advanced machine learning algorithms such as Random Forest, XGBoost, or Deep Learning models can be integrated to improve risk score accuracy.

- **Self-Learning Model:**

The system can be enhanced to learn from previous scan results and user feedback to continuously improve prediction performance.

3. Cloud & Enterprise Integration

- **Cloud Deployment:**

The system can be deployed on cloud platforms (AWS, Azure, or GCP) for scalable enterprise-level usage.

- **Multi-User Role Management:**

Future versions can include role-based access control (Admin, Security Analyst, Viewer) for organizational use.

- **API Integration:**

The system can provide public APIs for integration with existing enterprise security tools and DevOps pipelines.

4. Enhanced Visualization & Reporting

- **Interactive Dashboards:**

Advanced interactive dashboards with dynamic graphs and analytics can be implemented for better decision-making.

- **Automated Email Reports:**

The system can automatically send PDF risk reports via email after each scan.

- **Comparative Risk Analysis:**

Users can compare multiple applications' risk scores over time using trend analysis charts.

5. Security & Performance Optimization

- **Encryption & Secure Authentication:**

Implementation of JWT authentication, encrypted database storage, and secure API endpoints.

- **Performance Optimization:**

System performance can be enhanced to handle large-scale application datasets efficiently.

- **Mobile Application Version:**

A mobile app version of the dashboard can be developed for easy access and monitoring.

6.2 Conclusion:

The **AI App Risk Prediction System** has been successfully designed and implemented to analyze and evaluate the security risks of applications using intelligent risk assessment techniques. The system provides users with a structured platform to scan applications, generate risk scores, detect potential security issues, and download detailed PDF reports. By integrating a Machine Learning model with a Flask backend, Streamlit frontend, and SQLite database, the project delivers a complete end-to-end risk analysis solution.

The system effectively performs risk score generation, security issue identification, scan history storage, and graphical risk visualization. Features such as Risk Distribution graphs, Risk Bar Analysis, Confusion Matrix evaluation, and automated PDF report generation enhance usability and analytical capability. The modular architecture ensures smooth communication between frontend, backend API, database, and ML components, making the system reliable and scalable.

Testing results demonstrate that the application works efficiently with accurate risk prediction outputs and proper database management. The confusion matrix evaluation confirms acceptable prediction performance, while manual and system testing validate functionality across all modules.

In conclusion, the AI App Risk Prediction System provides a secure, intelligent, and user-friendly platform for application security analysis. It successfully meets the project objectives of predicting application risk levels, identifying security issues, and generating structured reports. With future enhancements such as advanced AI models and cloud deployment, the system has strong potential to evolve into a comprehensive enterprise-level cybersecurity solution.

7.REFERENCES

7.1 Refrence Websites

- Streamlit Official Documentation: <https://docs.streamlit.io/>
- Flask Official Documentation: <https://flask.palletsprojects.com/>
- FastAPI Official Documentation: <https://fastapi.tiangolo.com/>
- SQLite Official Documentation: <https://www.sqlite.org/docs.html>
- Scikit-learn Documentation (Confusion Matrix & ML tools): <https://scikit-learn.org/stable/>
- Matplotlib Documentation (Graphs & Visualization): <https://matplotlib.org/stable/index.html>
- Python Official Documentation: <https://docs.python.org/3/>

7.2 Refrence Books

- Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow – Aurélien Géron
- Python Machine Learning – Sebastian Raschka & Vahid Mirjalili
- Machine Learning: A Probabilistic Perspective – Kevin P. Murphy
- Clean Code: A Handbook of Agile Software Craftsmanship – Robert C. Martin
- Software Engineering – Ian Sommerville

7.3 Refrence Research paper

- A Survey on Machine Learning Techniques for Cyber Security Applications – IEEE Xplore
- Application Security Risk Assessment Using Machine Learning – Springer Research Paper
- Automated Vulnerability Detection Using Artificial Intelligence – International Journal of Computer Applications
- Risk-Based Security Analysis for Web Applications – ResearchGate Publication

8. GLOSSARY

1. Artificial Intelligence (AI)

Technology that enables machines to simulate human intelligence for tasks like prediction and decision-making.

2. Machine Learning (ML)

A subset of AI that allows systems to learn from data and improve performance without being explicitly programmed.

3. Risk Score

A numerical value (0–100) generated by the system indicating the security level of an application.

4. Risk Prediction Model

A trained machine learning model used to analyze application data and predict security risk levels (Low, Medium, High).

5. Backend API

The server-side application (Flask/FastAPI) that processes requests, runs the ML model, and returns results.

6. Frontend (Streamlit)

The user interface that allows users to enter application names, view risk scores, graphs, and download reports.

7. Database (SQLite)

A lightweight database used to store scan history, risk scores, and timestamps.

8. Confusion Matrix

A performance evaluation table used to measure the accuracy of the classification model.

9. PDF Report Generation

A feature that creates a downloadable security report containing risk score and detected issues.

10. Risk Distribution Graph

A visualization (pie chart) showing the percentage of Low, Medium, and High risk categories.

11. API Endpoint

A specific URL (e.g., /scan) used to send and receive data between frontend and backend.

12. Scan History

A record of previously scanned applications stored in the database.

13. Deployment

The process of making the system available for users to access and use.

14. Data Visualization

The graphical representation of risk scores and analysis results using charts and graphs.

15. Security Issues

Potential vulnerabilities detected by the system, such as weak encryption or hardcoded credentials.